

```
NAME: PRADIP KALE
ROLL_NO:13224
BATCH: B1
Contents for Theory:
7. Text Analytics
1. Extract Sample document and apply following document preprocessing
methods:
Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of document by calculating Term Frequency and
Inverse Document
Frequency
```

```
In [18]: import nltk
import re
```

```
In [19]: nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to C:\Users\soham
[nltk_data]   kakade\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\soham
[nltk_data]   kakade\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\soham
[nltk_data]   kakade\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to C:\Users\soham
[nltk_data]   kakade\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\soham kakade\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to[nltk_data]
date!
```

```
Out[19]: True
```

```
In [20]: text = "Tokenization is the first step in text analytics. It breaks text into
```

```
Filtered Words: ['Tokenization', 'first', 'step', 'text', 'analytics', '.',
'breaks', 'text', 'smaller', 'units', '.']
```

```
In [22]: from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))

filtered_tokens = []
for word in word_tokens:
    if word.lower() not in stop_words:
        filtered_tokens.append(word)

print("Filtered Words:", filtered_tokens)
```

```
In [23]: from nltk.stem import PorterStemmer

ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered_tokens]

print("Stemmed Words:", stemmed)

Stemmed Words: ['token', 'first', 'step', 'text', 'analyt', '.', 'break', 'te
xt', 'smaller', 'unit', '.']
```

```
In [24]: from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(w) for w in filtered_tokens]

print("Lemmatized Words:", lemmatized)

Lemmatized Words: ['Tokenization', 'first', 'step', 'text', 'analytics', '.',
'break', 'text', 'smaller', 'unit', '.']
```

```
In [8]: ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered_tokens]
print("\nStemmed Words:", stemmed)

Stemmed Words: ['text', 'analyt', 'process', 'analyz', 'text', 'data', 'extra
ct', 'use', 'inform', '.']
```

```
In [25]: pos_tags = nltk.pos_tag(filtered_tokens)
print("POS Tags:", pos_tags)

POS Tags: [('Tokenization', 'NN'), ('first', 'RB'), ('step', 'VB'), ('text',
'JJ'), ('analytics', 'NNS'), ('.', '.'), ('breaks', 'NNS'), ('text', 'JJ'),
('smaller', 'JJR'), ('units', 'NNS'), ('.', '.')]

```

```
In [26]: from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [27]: doc1 = "Jupiter is the largest planet"
doc2 = "Mars is the fourth planet from the sun"
```

```
In [28]: documents = [doc1, doc2]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(docs)
```

```
In [29]: print("Words:", vectorizer.get_feature_names_out())
```

```
Words: ['analysis' 'analytics' 'analyzing' 'and' 'are' 'data' 'extracts' 'fro
m'
'important' 'information' 'is' 'mining' 'of' 'process' 'text' 'the'
'useful']
```

```
In [30]: import pandas as pd
tfidf_df = pd.DataFrame(X.toarray(), columns=features)
```

```
In [31]: print("\nTF-IDF Matrix:\n")
print(tfidf_df)
```

TF-IDF Matrix:

	analysis	analytics	analyzing	and	are	data	extracts	\
0	0.000000	0.276745	0.363886	0.000000	0.000000	0.276745	0.000000	
1	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.39543	
2	0.426184	0.324124	0.000000	0.426184	0.426184	0.324124	0.000000	

	from	important	information	is	mining	of	process	\
0	0.000000	0.000000	0.000000	0.363886	0.000000	0.363886	0.363886	
1	0.39543	0.000000	0.39543	0.000000	0.39543	0.000000	0.000000	
2	0.000000	0.426184	0.000000	0.000000	0.000000	0.000000	0.000000	

	text	the	useful	0
	0.429834	0.363886	0.000000	
1	0.467094	0.000000	0.39543	
2	0.251711	0.000000	0.000000	

```
In [ ]:
```